INF7370 Apprentissage automatique – Hiver 2026
Travail pratique #1
Professeur : Mohamed Bouguessa
Envoyé le 26 janvier 2026
Date de remise : ~~le 26 février 2026 à 09h30~~

le 9 mars 2026 à 9h30

Consignes importantes

1. **Notation :** Le TP est noté sur 23 points.
2. **Travail en équipe :** Vous pouvez réaliser ce travail individuellement ou en équipe de deux étudiant(e)s maximum.
3. **Fichiers à soumettre :**
   - **Données d'apprentissage :** Le fichier CSV contenant l'ensemble d'apprentissage que vous avez préparé. **Ne pas remettre les fichiers de données brutes.**
   - **Code source :**
     - Soumettez vos fichiers Python uniquement. Les fichiers Jupyter Notebook ne sont pas acceptés.
     - Donnez des noms pertinents à vos fichiers. Par exemple :
       - features_extraction.py (pour le calcul des attributs)
       - comparison_all_algorithms.py (pour la comparaison des algorithmes)
       - imbalanced_data_comparison.py (pour la comparaison sur les données non équilibrées).
     - Fournissez un code bien organisé, avec une décomposition claire des tâches dans différents fichiers.
     - Ajoutez des commentaires explicatifs pour chaque fonction que vous implémentez.
     - Incluez un fichier texte (ex. : Readme.txt) pour aider le correcteur à exécuter votre code.
   - **Rapport :**
     - Votre rapport doit être en format PDF et identifier clairement votre nom.
     - Suivez les directives de rédaction du rapport.
   - **Fichier compressé :**
     - Regroupez tous les fichiers ci-dessus dans un **unique fichier ZIP** identifié à votre nom. N'utilisez pas de noms génériques comme "TP1".
     - **Format de soumission obligatoire : ZIP uniquement.**
     - Assurez-vous que vos fichiers s'ouvrent sans problème.
   - Une seule soumission par équipe est permise.

4. **Présentation et rigueur :**
    - Prenez soin de la présentation et de la rédaction de votre rapport.
    - Les résultats doivent être présentés sous forme de graphiques ou de tableaux bien organisés. **Ne soumettez pas de captures d'écran brutes.**
    - Ne faites pas une analyse sommaire, générique ou superficielle.
5. **Retard et soumission :**
    - La date limite est le ~~26 février 2026 à 09h30~~. 9 mars 2026 à 9h30.
    - Moodle ferme automatiquement à cette heure. Aucun travail ne sera accepté après, quelle que soit la raison (y compris les problèmes techniques).
    - La soumission se fait uniquement via Moodle. **Aucune soumission par courriel ne sera corrigée.**

Objectifs du TP

Ce travail pratique a pour objectif d'analyser et de comparer les performances de plusieurs algorithmes de classification à l'aide de données collectées sur X (Twitter). Les algorithmes à étudier sont :

1. Arbre de décision
2. Bagging
3. AdaBoost
4. Boosting de gradient (GBoost)
5. Boosting de gradient extrême (XGBoost)
6. Forêts d'arbres aléatoires
7. Classification bayésienne naïve

Les données se composent de tweets provenant de deux catégories d'utilisateurs :

1. **Content Polluters** (pollueurs)
2. **Legitimate Users** (légitimes)

Les fichiers texte contenant ces données (six fichiers au total) sont disponibles sur Moodle. Ces fichiers incluent :

- 2 353 473 tweets postés par 22 223 utilisateurs pollueurs.
- 3 259 693 tweets postés par 19 276 utilisateurs légitimes.

Votre travail consiste à analyser ces données et à comparer les performances des algorithmes mentionnés.

Tâches à réaliser

Tâche 1 : Compréhension de l'algorithme XGBoost

- Expliquez le principe de fonctionnement de XGBoost (eXtreme Gradient Boosting).
- Discutez ses principaux apports par rapport au boosting de gradient classique.
- Mettez en avant ses points forts et ses avantages par rapport à d'autres algorithmes.
- La description doit couvrir **au moins une page.**

Tâche 2 : Préparation des données

1. **Calcul des caractéristiques :** Implémentez les caractéristiques suivantes pour chaque utilisateur :
    - Longueur du nom d'utilisateur
    - Longueur de la description du profil
    - Durée de vie du compte
    - Nombre de following
    - Nombre de followers
    - Rapport « following/followers »
    - Nombre moyen de tweets par jour
    - Proportion d'URL dans les tweets
    - Proportion de mentions @ dans les tweets
    - Temps moyen et maximal entre deux tweets consécutifs
    - **Deux caractéristiques supplémentaires :** Proposez et implémentez deux nouvelles caractéristiques pertinentes.

2. **Préparation de l'ensemble d'apprentissage :**
    - Nettoyez les données :
        - Suppression des doublons
        - Traitement des valeurs manquantes (ex. : remplacement par la médiane)
        - Normalisation des données (ex. : z-score)
    - Regroupez les données dans un fichier CSV avec une colonne indiquant la classe (1 : pollueur, 0 : légitime).

Tâche 3 : Analyse comparative

- Appliquez les sept algorithmes : Arbre de décision, Bagging, AdaBoost, GBoost, XGBoost, Forêts aléatoire, bayésien naïf.
- Comparez leurs performances en utilisant les métriques suivantes :
    - Taux de vrais positifs (TP Rate) pour les pollueurs
    - Taux de faux positifs (FP Rate) pour les pollueurs
    - F-mesure pour les pollueurs
    - Aire sous la courbe ROC (AUC)
- Présentez les résultats sous forme de graphiques et tableaux.
- Analysez les points forts et faibles de chaque algorithme (pas d'analyse générique/superficielle).

Tâche 4 : Analyse comparative sur des classes déséquilibrées

- À partir des données issues de la Tâche 2, créez un sous-ensemble où les pollueurs représentent 5 % des utilisateurs légitimes.
- Refaites l'analyse de la Tâche 3.
- Comparez les résultats avec ceux obtenus avec les données équilibrées.

Bibliothèques qui peuvent être utilisées :

1. **Scikit-Learn** : Bibliothèque libre et open source implémentée avec Python, dédié à l'apprentissage automatique. Elle est conçue pour s'harmoniser avec d'autres bibliothèques Python (ou autres), notamment NumPy, SciPy, etc.

2. **Scikit-plot** : Bibliothèque libre et open source implémentée avec Python, utilisé pour l'affichage des graphiques principalement les courbes ROC et AUC.

3. **Numpy** : "Numerical Python" fournit une interface pour stocker et effectuer des opérations sur les données. D'une certaine manière, les tableaux Numpy sont comme les listes en Python, mais Numpy permet de rendre les opérations beaucoup plus efficaces, surtout sur les tableaux de grande taille qui sont au cœur de l'écosystème de la Data Science.

4. **Pandas** : Fournis deux structures de données fondamentales, la "Série" et le "DataFrame". On peut voir ces structures comme une généralisation des tableaux et des matrices de Numpy. La différence entre les structures de Pandas et celles de Numpy c'est la définition explicite par l'utilisateur des indices et des index sur les objets (matrices).

5. **Pickle** : Module Python utilisé pour sérialiser et dé-sérialiser les structures d'objets Python. La sérialisation (ou « pickling ») fait référence au processus de conversion d'un objet en mémoire en un flux d'octets pouvant être stocké sur disque ou envoyé sur un réseau. Plus tard, ce flux de caractères peut être récupéré et dé-sérialiser (ou «unpickling») en retour vers un objet Python.

6. **Matplotlib** : "Mathematic Plot library" est une bibliothèque de traçage Python 2D qui produit des figures de qualité de publication dans une variété de formats papier et d'environnements interactifs entre plates-formes. Matplotlib peut être utilisé dans les scripts Python, les Shells Python et IPython, les notebook Jupyter, etc.